

Jenkins 4 second doc task (2-6)

Tasks

2) Setup a jenkins CICD pipeline using Declarative pipeline using feature-1.1 branch.

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

- 1) Git Clone
- 2) Sonarqube Integration
- 3) Maven Compilation
- 4) Nexus Artifactory
- 5) Slack Notification
- 6) Deploy On tomcat

Jenkins Setup for Your Jenkinsfile

1. Install Required Plugins

Go to **Manage Jenkins** → **Plugins** and ensure these are installed:

- **Pipeline** (Declarative + Scripted)
- **Pipeline Utility Steps** (for readMavenPom, findFiles)
- **Nexus Artifact Uploader**
- **SonarQube Scanner for Jenkins**
- **Slack Notification**

2. Configure Global Tools

Manage Jenkins → Global Tool Configuration

- **Maven:**
 - Name: MVN_HOME (matches your Jenkinsfile)
 - Install automatically or point to /usr/share/maven
- **SonarQube Scanner:**
 - Name: sonar_scanner (matches your Jenkinsfile)
 - Provide installation path or let Jenkins auto-install

3. Add Server Integrations

a) SonarQube

- Go to **Manage Jenkins → Configure System → SonarQube servers**
 - Name: sonarqube-server (matches your Jenkinsfile)
 - Add Server URL (e.g., http://<sonar-host>:9000)
 - Add authentication token (create in SonarQube UI → My Account → Security → Tokens)

b) Slack

- Go to **Manage Jenkins → Configure System → Slack**
 - Workspace: your Slack team domain
 - Integration token (from Slack App)
 - Default channel: #jenkins-integration
- Test connection to ensure Jenkins can post.

4. Set Credentials

Manage Jenkins → Credentials → System → Global credentials

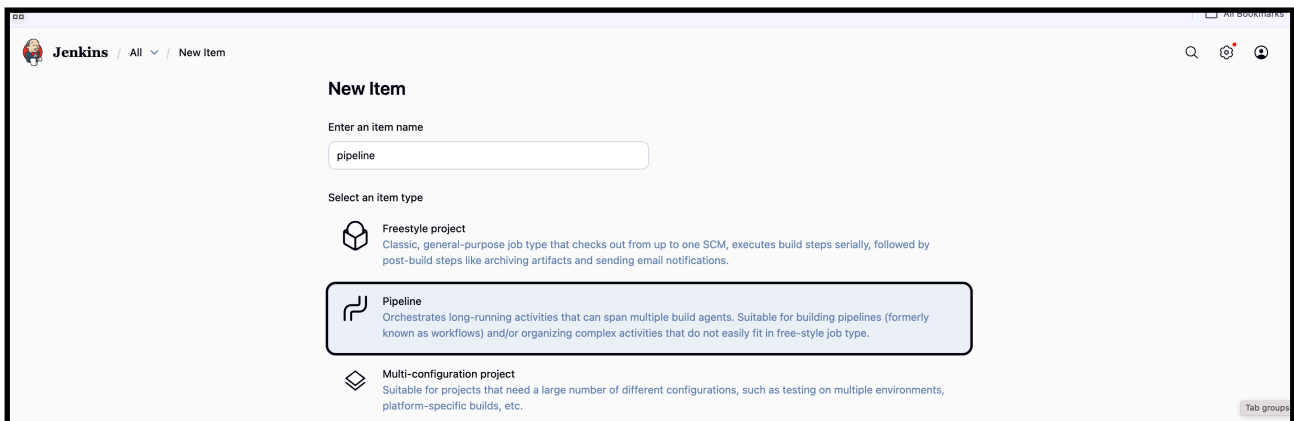
Add the following IDs (must match your Jenkinsfile):

- **Nexus-server**
 - Type: Username with password
 - Username: Nexus repo user
 - Password: Nexus repo password
- **tomcat**
 - Type: Username with password

- Username: Tomcat manager user
- Password: Tomcat manager password

5. Create a New Pipeline Job

1. Open Jenkins dashboard → **New Item**.
2. Enter a job name (e.g., simple-customerapp-pipeline).
3. Select **Pipeline** → Click **OK**.



Drop this Jenkins pipeline

Stages: **Git Clone** → **SonarQube** → **Maven Compile** → **Nexus Upload** → **Deploy on Tomcat** → **Slack**

Pipeline code :

```

pipeline {
    agent any

    environment {
        MAVEN_HOME = tool 'MVN_HOME'
        SONAR_HOME = tool 'sonar_scanner'

        SONAR_PROJECT_KEY = 'Sabear'
        SONAR_PROJECT_NAME = 'Sabear'

        NEXUS_URL = 'http://44.211.69.235:8081'
        NEXUS_REPO = 'hiring-app'
        NEXUS_CRED = 'nexus-user'

        TOMCAT_URL = 'http://44.199.202.221:8080'
        TOMCAT_CONTEXT = 'simplecustomerapp'
        TOMCAT_CRED = 'tomcat_credentials'
    }

    stages {

        stage('Git Clone') {
            steps {
                git 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
            }
        }

        stage('Maven Build') {
            steps {
                sh """
                ${MAVEN_HOME}/bin/mvn clean package -Dmaven.test.failure.ignore=true
                """
            }
        }
    }
}

```



```

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('sonarqube-server') {
            sh """
            ${SONAR_HOME}/bin/sonar-scanner \
            -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
            -Dsonar.projectName=${SONAR_PROJECT_NAME} \
            -Dsonar.sources=src \
            -Dsonar.exclusions=**/*.java
            """
        }
    }
}

```

```

stage('Upload WAR to Nexus') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: "${NEXUS_CRED}",
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASS'
        )]) {
            sh """
            WAR_FILE=$(ls target/*.war | head -n 1)

            curl -u $NEXUS_USER:$NEXUS_PASS \
            --upload-file $WAR_FILE \
            http://44.211.69.235:8081/repository/hiring-app/in/javahome/hiring/0.1/hiring-0.1.war
            """
        }
    }
}

```

```

stage('Deploy to Tomcat') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: "${TOMCAT_CRED}",

```

```

        usernameVariable: 'TOMCAT_USER',
        passwordVariable: 'TOMCAT_PASS'
    )) {
        sh ""
        WAR_FILE=$(ls target/*.war | head -n 1)

        curl -u $TOMCAT_USER:$TOMCAT_PASS \
        -T $WAR_FILE \
        "${TOMCAT_URL}/manager/text/deploy?path=/${TOMCAT_CONTEXT}
&update=true"
        ""
    }
}
}

post {
    success {
        echo "✅ PIPELINE SUCCESS: BUILD → SONAR → NEXUS → TOMCAT"
    }
    failure {
        echo "❌ PIPELINE FAILED"
    }
}
}

```

Jenkins

pipeline

Configuration

Configure

General

Triggers

Pipeline

Advanced

Pipeline script

Script

```
1 pipeline {
2   agent any
3
4   environment {
5     // ===== TOOLS =====
6     MAVEN_HOME = tool 'MVN_HOME'
7     SONAR_HOME = tool 'sonar_scanner'
8
9     // ===== SONAR =====
10    SONAR_PROJECT_KEY = "Sabear"
11    SONAR_PROJECT_NAME = "Sabear"
12
13    // ===== NEXUS =====
14    NEXUS_URL = "http://13.221.106.196:8081"
15    NEXUS_REPO = "hiring-app"
16  }
17 }
```

☒ Use Groovy Sandbox

[Pipeline Syntax](#)

Advanced

Advanced

Jenkins

pipeline

#36

Console Output

Status

Changes

Console Output

Edit Build Information

Delete build '#36'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console Output

Download

Copy

View as plain text

Started by user waseem akram

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in /var/lib/jenkins/workspace/pipeline

[Pipeline] {

[Pipeline] tool

[Pipeline] tool

[Pipeline] withEnv

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Git Clone)

[Pipeline] git

Selected Git installation does not exist. Using Default

The recommended git tool is: NONE

No credentials specified

> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/pipeline/.git # timeout=10

Fetching changes from the remote Git repository

> git config remote.origin.url https://github.com/Waseema761/sabear_simplecutomerapp.git # timeout=10

Fetching upstream changes from https://github.com/Waseema761/sabear_simplecutomerapp.git

> git --version # timeout=10

> git --version # 'git version 2.50.1'

> git fetch --tags --force --progress -- https://github.com/Waseema761/sabear_simplecutomerapp.git +refs/heads/*:refs/remotes/origin/* # timeout=10

> git rev-parse refs/remotes/origin/master^{commit} # timeout=10

Checking out Revision f386c3ea540af6aff21dd0c7f2c55de7d31d5219 (refs/remotes/origin/master)

> git config core.sparsecheckout # timeout=10

> git checkout -f f386c3ea540af6aff21dd0c7f2c55de7d31d5219 # timeout=10

> git branch -a -v --no-abbrev # timeout=10

> git branch -D master # timeout=10

> git checkout -b master f386c3ea540af6aff21dd0c7f2c55de7d31d5219 # timeout=10

Commit message: "Refactor Jenkinsfile for improved structure and clarity"

> git rev-list --no-walk f386c3ea540af6aff21dd0c7f2c55de7d31d5219 # timeout=10

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

Jenkins / pipeline / #36 / Pipeline Overview

Manually run by waseem akram Started 6 min 49 sec ago Queued 1 ms Took 10 sec

Graph

Start Git Clone Maven Build SonarQube Analysis Upload WAR to Nexus Deploy to Tomcat Post Actions End

Post Actions 41ms Started 6m 40s ago Jenkins

Git Clone 0.13s

Maven Build 2.6s

SonarQube Analysis 5.9s

Upload WAR to Nexus 0.35s

Deploy to Tomcat 0.35s

Post Actions 41ms

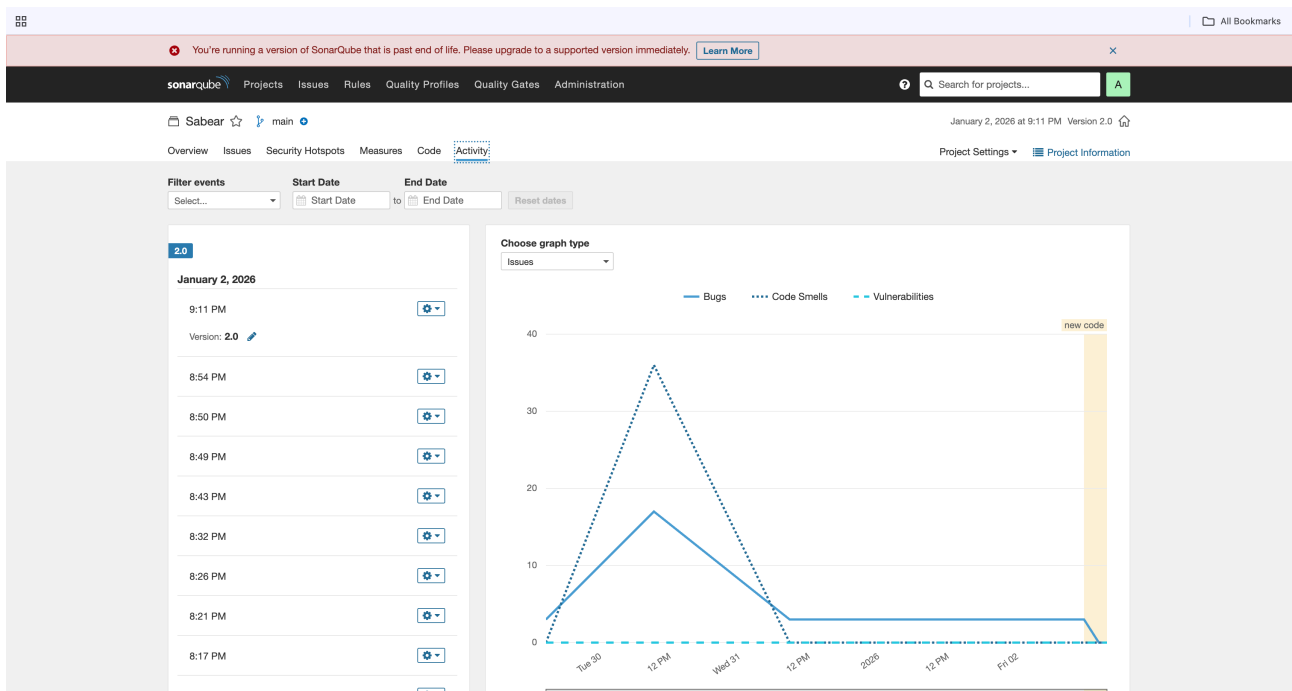
PIPELINE SUCCESS: BUILD → SONAR → NEXUS → TOMCAT

← → ↻ ⚠ Not Secure 3.238.239.27:8081/job/pipeline/36/pipeline-overview/log?nodeId=48

Click to go back, hold to see history

PIPELINE SUCCESS: BUILD → SONAR → NEXUS → TOMCAT

—>Sonarqube Integration



Nexus Artifactory

The screenshot shows the Sonatype Nexus Repository OSS 3.68.1-02 interface. The left sidebar has a 'Browse' section with a search bar and a list of components. The main content area displays the 'Browse' view for the component 'in/javahome/hiring / hiring-app'. The 'HTML View' shows a tree structure of the component, with 'hiring -0.1.war' selected. The right sidebar shows the 'Summary' of the selected component, including details like Repository, Format, Component Group, Component Name, Component Version, Path, Content type, File size, Blob created, Blob updated, Last downloaded, Locally cached, Blob reference, Containing repo, Uploader, and Uploader's IP Address.

Property	Value
Repository	hiring-app
Format	maven2
Component Group	in.javahome
Component Name	hiring
Component Version	0.1
Path	in/javahome/hiring /0.1/hiring -0.1.war
Content type	application/java-archive
File size	1.7 KB
Blob created	Fri Jan 02 2026 15:26:47 GMT+0530 (India Standard Time)
Blob updated	Fri Jan 02 2026 17:13:34 GMT+0530 (India Standard Time)
Last downloaded	has not been downloaded
Locally cached	true
Blob reference	default@29ced931-f2c0-43c8-a1a0-72064f0538ca
Containing repo	hiring-app
Uploader	admin
Uploader's IP Address	3.238.239.27

Deploy On tomcat

```
root@ip-172-31-10-35 hiring]# cd ..
[root@ip-172-31-10-35 webapps]# ls
ROOT docs examples hiring hiring.war host-manager manager simplecustomerapp simplecustomerapp.war
[root@ip-172-31-10-35 webapps]# cd simplecustomerapp.war
-bash: cd: simplecustomerapp.war: Not a directory
[root@ip-172-31-10-35 webapps]# cd simplecustomerapp.war
-bash: cd: simplecustomerapp.war: Not a directory
[root@ip-172-31-10-35 webapps]# cd simplecustomerapp/
[root@ip-172-31-10-35 simplecustomerapp]# ls
META-INF WEB-INF
[root@ip-172-31-10-35 simplecustomerapp]#
```

Check on the web with the public ip and the deployed file name

3. Setup a Jenkins CI/CD pipeline using Scripted pipeline using feature-1.1 branch. https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

Git Clone

SonarQube Integration

Maven Compilation

Nexus Artifactory

Slack Notification

Deploy On tomcat

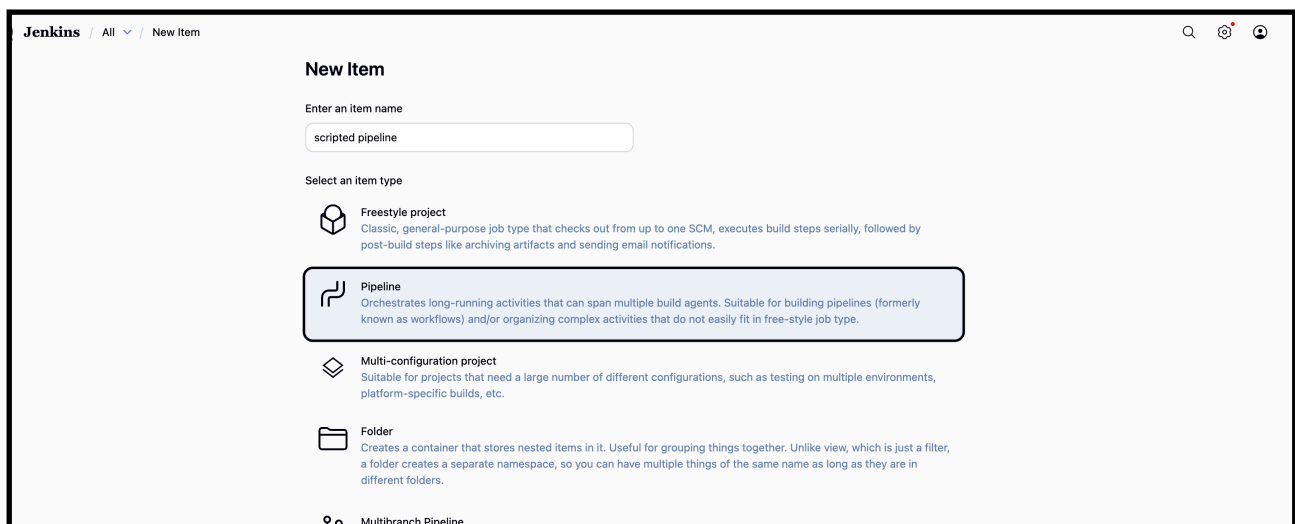
Steps to Create a Scripted Pipeline in Jenkins

1. Log in to Jenkins

- Open Jenkins in your browser (<http://<jenkins-server>:8080/>).
- Log in with your credentials.

2. Create a New Pipeline Job

- From Jenkins Dashboard → Click **“New Item”**.
- Enter a **name** for your job (e.g., My-Scripted-Pipeline).
- Select **Pipeline** and click **OK**.



3. Configure Source Code Management (SCM)

- In the job configuration:
 - Go to **Pipeline → Definition**.
 - Choose one of the two approaches:
 - **Pipeline script from SCM** (if your Jenkinsfile is already in your repo).
 - **Pipeline script** (if you want to write or paste the Scripted Pipeline directly in Jenkins).

4. Scripted Pipeline Structure

A **Scripted Pipeline** uses Groovy syntax.

The screenshot shows the Jenkins Configuration page for a 'scripted pipeline'. The left sidebar contains a 'Configure' section with tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced'. The 'Pipeline' tab is selected. The main configuration area is titled 'Definition' and shows 'Pipeline script from SCM' selected in a dropdown. Below this, the 'SCM' section is set to 'Git'. A 'Repositories' section contains one repository with the URL 'https://github.com/Waseema761/sabear_simplecouterapp.git' and credentials set to '- none -'. There is an 'Add Repository' button. The 'Branches to build' section has a 'Branch Specifier (blank for \'any\')' set to '*/feature1' and an 'Add Branch' button. The 'Repository browser' section is empty. At the bottom, there are 'Save' and 'Apply' buttons. The bottom left corner shows the IP address '8.82.78:8081'.

Jenkins / scripted pipeline / Configuration

Configure

General
Triggers
Pipeline
Advanced

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Waseema761/sabear_simplecouterapp.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/feature1

+ Add Branch

Repository browser ?

Save Apply

8.82.78:8081

This is a close-up view of the Jenkins Configuration page. It shows the 'Branch Specifier (blank for \'any\')' field with the value '*/feature1' and an 'Add Branch' button. Below this is the 'Repository browser' dropdown set to '(Auto)'. The 'Additional Behaviours' section has an 'Add' button. The 'Script Path' field is set to 'Jenkinsfile'. The 'Lightweight checkout' checkbox is checked. A link for 'Pipeline Syntax' is visible. The 'Advanced' section is partially visible at the bottom.

Branch Specifier (blank for 'any') ?

*/feature1

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

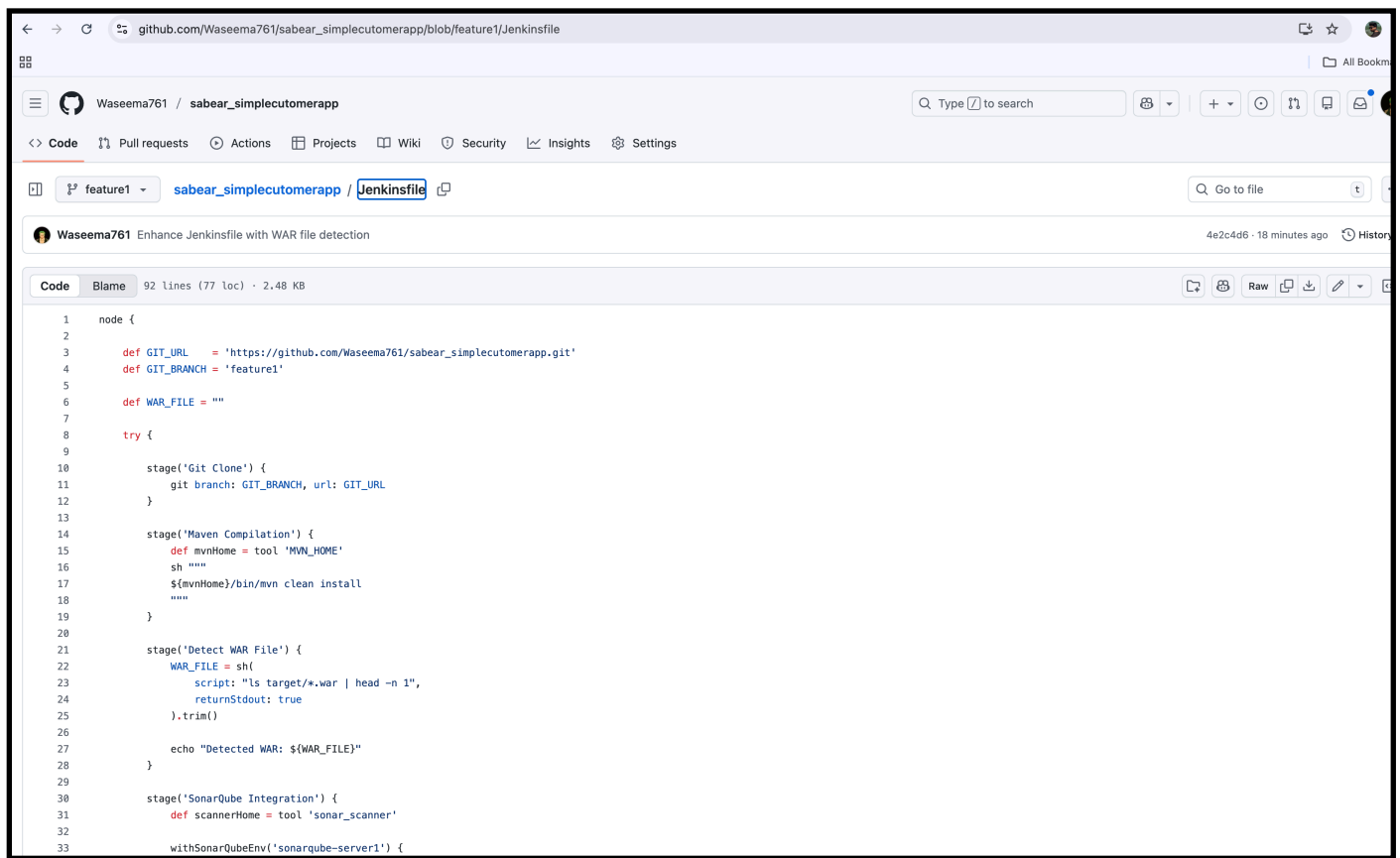
Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Advanced

—> edit the file and add the code inside



The screenshot shows a GitHub repository page for 'Waseema761 / sabear_simplecutomerapp'. The 'Jenkinsfile' is selected in the file browser. The commit message is 'Enhance Jenkinsfile with WAR file detection'. The code is displayed in a light-themed editor with line numbers 1 through 33. The code defines variables for Git URL, branch, and WAR file, and includes stages for cloning, Maven compilation, WAR file detection, and SonarQube integration.

```
1 node {
2
3   def GIT_URL = 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
4   def GIT_BRANCH = 'feature1'
5
6   def WAR_FILE = ""
7
8   try {
9
10    stage('Git Clone') {
11      git branch: GIT_BRANCH, url: GIT_URL
12    }
13
14    stage('Maven Compilation') {
15      def mvnHome = tool 'MVN_HOME'
16      sh """
17        ${mvnHome}/bin/mvn clean install
18      """
19    }
20
21    stage('Detect WAR File') {
22      WAR_FILE = sh(
23        script: "ls target/*.war | head -n 1",
24        returnStdout: true
25      ).trim()
26
27      echo "Detected WAR: ${WAR_FILE}"
28    }
29
30    stage('SonarQube Integration') {
31      def scannerHome = tool 'sonar_scanner'
32
33      withSonarQubeEnv('sonarqube-server1') {
```

—> below is the code I have written to build success

```

1 node {
2
3     def GIT_URL = 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
4     def GIT_BRANCH = 'feature1'
5
6     def WAR_FILE = ""
7
8     try {
9
10        stage('Git Clone') {
11            git branch: GIT_BRANCH, url: GIT_URL
12        }
13
14        stage('Maven Compilation') {
15            def mvnHome = tool 'MVN_HOME'
16            sh """
17            ${mvnHome}/bin/mvn clean install
18            """
19        }
20
21        stage('Detect WAR File') {
22            WAR_FILE = sh(
23                script: "ls target/*.war | head -n 1",
24                returnStdout: true
25            ).trim()
26
27            echo "Detected WAR: ${WAR_FILE}"
28        }
29
30        stage('SonarQube Integration') {
31            def scannerHome = tool 'sonar_scanner'
32
33            withSonarQubeEnv('sonarqube-server1') {

```

```

        stage('Maven Compilation') {
            def mvnHome = tool 'MVN_HOME'
            sh """
            ${mvnHome}/bin/mvn clean install
            """
        }

        stage('Detect WAR File') {
            WAR_FILE = sh(
                script: "ls target/*.war | head -n 1",
                returnStdout: true
            ).trim()

            echo "Detected WAR: ${WAR_FILE}"
        }

        stage('SonarQube Integration') {
            def scannerHome = tool 'sonar_scanner'

            withSonarQubeEnv('sonarqube-server1') {
                sh """
                ${scannerHome}/bin/sonar-scanner \
                -Dsonar.projectKey=Sabear \
                -Dsonar.projectName=Sabear \
                -Dsonar.projectVersion=1.0 \
                -Dsonar.sources=src \
                -Dsonar.java.binaries=target/**/WEB-INF/classes
                """
            }
        }

        stage('Nexus Artifactory') {
            nexusArtifactUploader(
                nexusVersion: 'nexus3',
                protocol: 'http',
                nexusUrl: '13.221.106.196:8081',
                repository: 'hiring-app',
                credentialsId: 'nexus-user',
                groupId: 'in.javahome',
                version: env.BUILD_NUMBER,
                artifacts: [[

```

```

        artifactId: 'SimpleCustomerApp',
        classifier: '',
        file: WAR_FILE,
        type: 'war'
    })
}

stage('Deploy On Tomcat') {
    deploy adapters: [
        tomcat9(
            credentialsId: 'tomcat_credentials',
            url: 'http://44.204.98.178:8080'
        )
    ],
    contextPath: 'SimpleCustomerApp',
    war: WAR_FILE
}

stage('Slack Notification') {
    slackSend(
        channel: '#jenkins-integration',
        color: 'good',
        message: "✅ BUILD SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    )
}

} catch (err) {

    slackSend(
        channel: '#jenkins-integration',
        color: 'danger',
        message: "❌ BUILD FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    )

    throw err
}
}

```

—> and save commit the changes

5. Save and Build

- Click **Save**.
- Run the job → Click **Build Now**.
- Check console output to verify pipeline execution



Status

Changes

Build Now

Configure

Delete Pipeline

SonarQube

Stages

Rename

Pipeline Syntax

✓ scripted pipeline

SonarQube Quality Gate

Sabear **Passed**

server-side processing: **Success**

Permalinks

- [Last build \(#10\), 21 min ago](#)
- [Last stable build \(#10\), 21 min ago](#)
- [Last successful build \(#10\), 21 min ago](#)
- [Last failed build \(#9\), 23 min ago](#)
- [Last unsuccessful build \(#9\), 23 min ago](#)
- [Last completed build \(#10\), 21 min ago](#)

Builds

Filter

Today

✓ #10 1:32 PM

✗ #9 1:30 PM

✗ #8 1:19 PM

✗ #7 1:14 PM

Status

Changes

Console Output

Edit Build Information

Delete build '#10'

Timings

Git Build Data

Pipeline Overview

Replay

Pipeline Steps

Workspaces

Previous Build

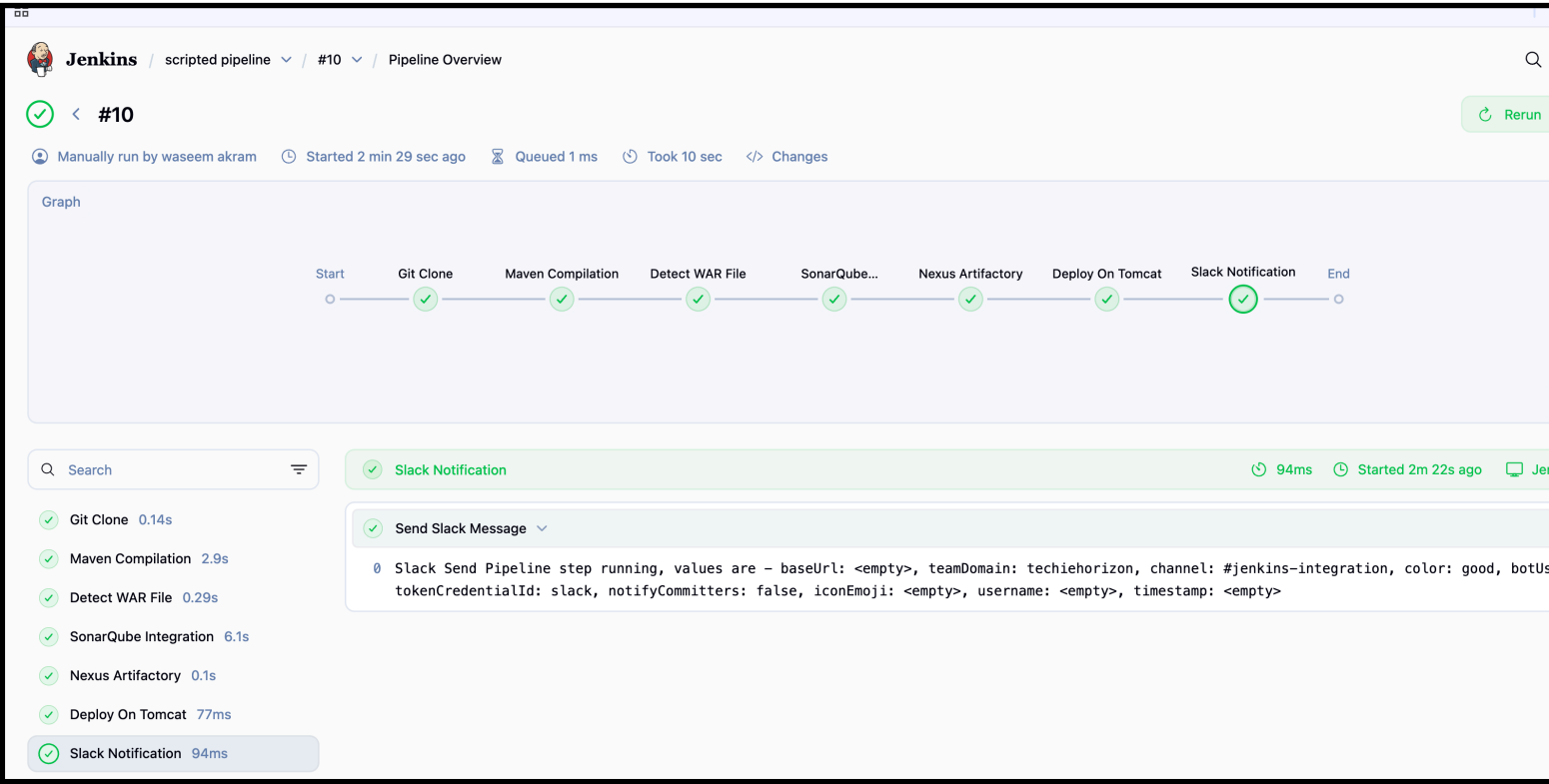
✓ Console Output

Download

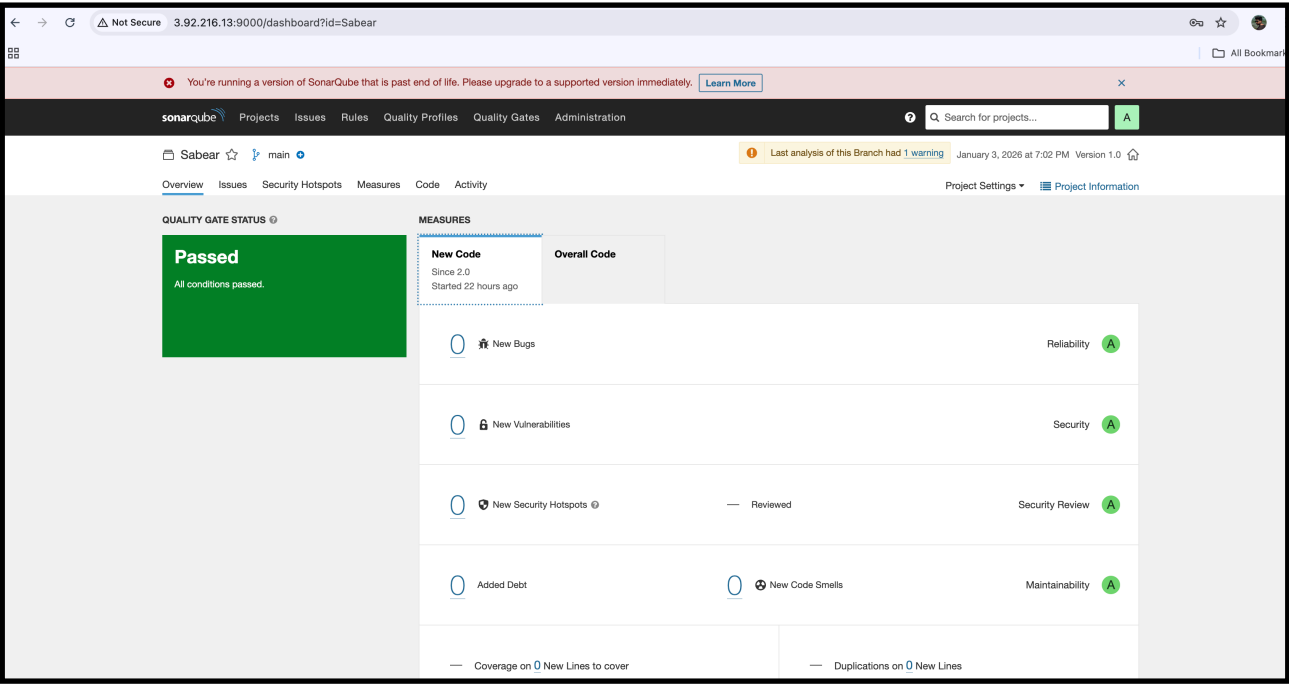
Copy

View as plain text

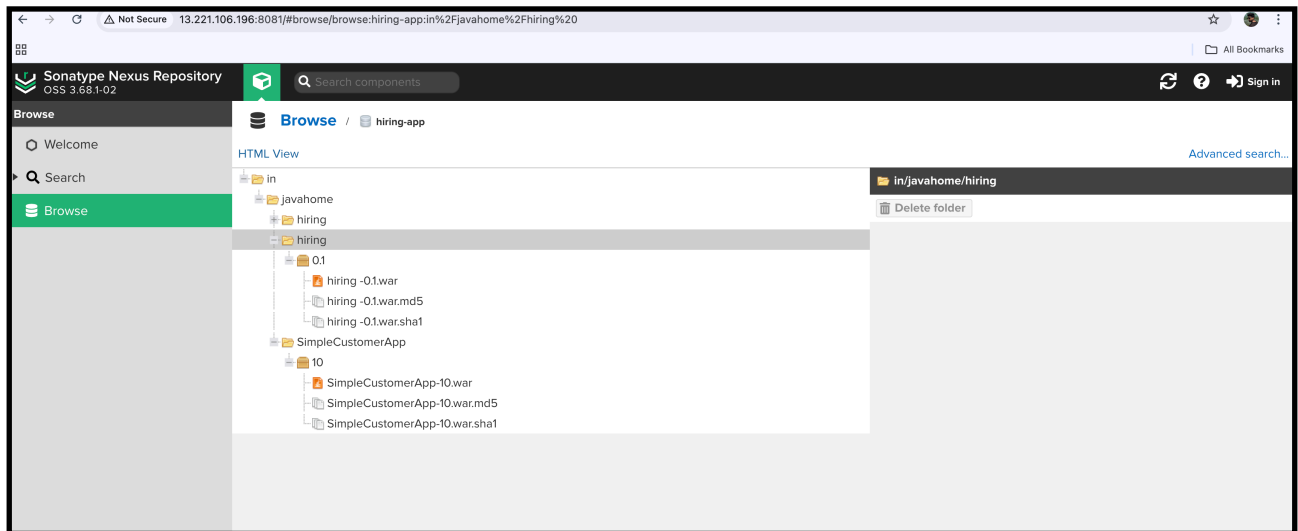
```
Started by user waseem akram
Obtained Jenkinsfile from git https://github.com/Waseema761/sabear_simplecutomerapp.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/scripted pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/scripted pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/sabear_simplecutomerapp.git # timeout=10
Fetching upstream changes from https://github.com/Waseema761/sabear_simplecutomerapp.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/sabear_simplecutomerapp.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/feature1^{commit} # timeout=10
Checking out Revision 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 (refs/remotes/origin/feature1)
> git config core.sparsecheckout # timeout=10
> git checkout -f 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D feature1 # timeout=10
> git checkout -b feature1 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 # timeout=10
Commit message: "Enhance Jenkinsfile with WAR file detection"
> git rev-list --no-walk dd7a8214f4eb022edfb92a77c682368a2cd33204 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Maven Compilation)
```



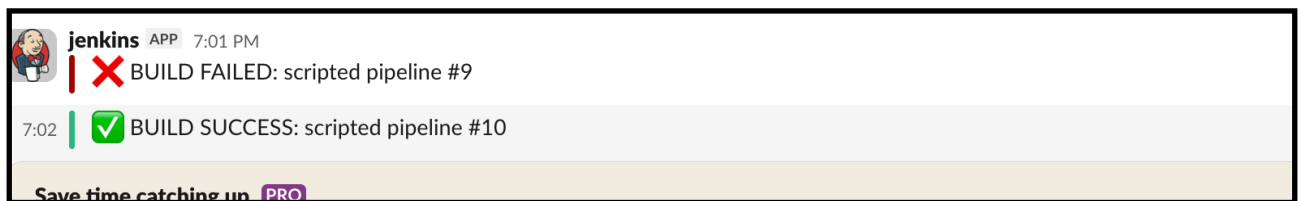
SonarQube Integration



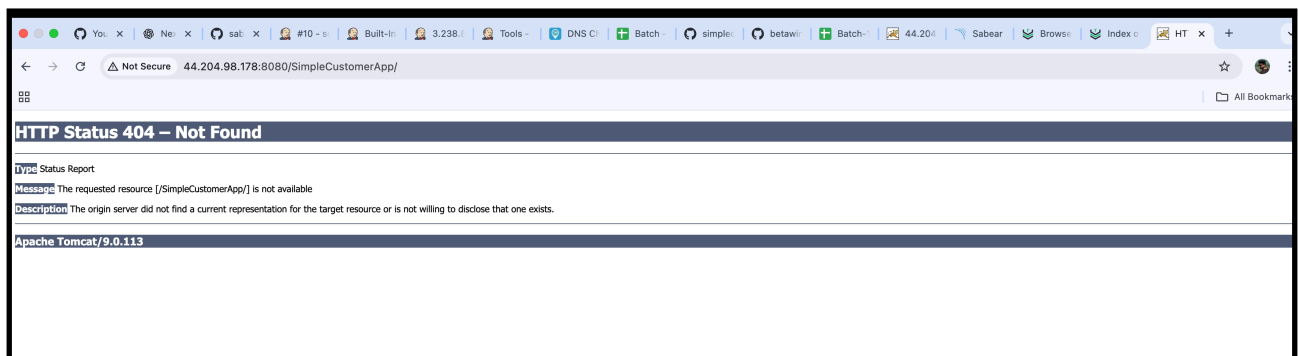
Nexus Artifactory



Slack Notification



Deploy On tomcat



4) Write sample skeleton of pipelines.

Declarative Pipeline Skeleton

1) Declarative Pipeline – Skeleton

```
pipeline {
    agent any

    stages {

        stage('Git Clone') {
            steps {
                git 'https://github.com/your-repo/sample-app.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('sonarqube-server') {
                    sh 'sonar-scanner'
                }
            }
        }

        stage('Deploy') {
            steps {
                sh 'cp target/*.war /opt/tomcat/webapps/'
            }
        }
    }
}
```

```

}

post {
    success {
        echo 'Build Successful'
    }
    failure {
        echo 'Build Failed'
    }
}
}
}

```

Conclusion for **Declarative** : Declarative pipeline uses a structured and predefined syntax, making it easier to read and manage.

2) Scripted Pipeline – Skeleton

```

node {

    stage('Git Clone') {
        git 'https://github.com/your-repo/sample-app.git'
    }

    stage('Build') {
        sh 'mvn clean install'
    }

    stage('SonarQube Analysis') {
        withSonarQubeEnv('sonarqube-server') {
            sh 'sonar-scanner'
        }
    }

    stage('Deploy') {
        sh 'cp target/*.war /opt/tomcat/webapps/'
    }
}

```


}

Conclusion for scripted :

Scripted pipeline uses Groovy scripting and provides more flexibility and control.

Scripted vs Declarative — stage-by-stage & config differences

(Your earlier Declarative pipeline → now Scripted)

- **Overall structure**
 - *Declarative*: pipeline { agent any; tools { ... } environment {...} stages {...} post {...} }
 - *Scripted*: imperative node { ... } + stage("...") { ... }. You call tool to resolve installations and assign to variables (e.g., mvnHome = tool 'MVN_HOME').
- **Tools**
 - *Declarative*: tools { maven 'MVN_HOME' } — Jenkins injects tool into PATH automatically for that node.
 - *Scripted*: def mvnHome = tool name: 'MVN_HOME', type: 'maven' — you must reference the full path \${mvnHome}/bin/mvn.
- **Environment**
 - *Declarative*: environment { VAR = 'value' } — automatically available to all steps.
 - *Scripted*: env.VAR = 'value' or assign local def variables. Use withCredentials to temporarily set credentials.
- **Post / Notifications**
 - *Declarative*: built-in post { success { ... } failure { ... } } blocks for cleanup/notifications.
 - *Scripted*: you must implement your own try/catch/finally or place slackSend stages explicitly.
- **Quality Gate**
 - *Declarative*: you usually use waitForQualityGate abortPipeline: true inside a stage.
 - *Scripted*: use def qg = waitForQualityGate() and check qg.status to decide to error.

- **Readability & Safety**
 - *Declarative*: enforces structure (agent, stages) and is easier for linters and shared libraries.
 - *Scripted*: more flexible/imperative — useful for complex logic, but easier to introduce hard-to-debug flows.
- **Credentials handling**
 - Same in both: use withCredentials([...]) in scripted; declarative supports credentials directive in environment as well.
 -

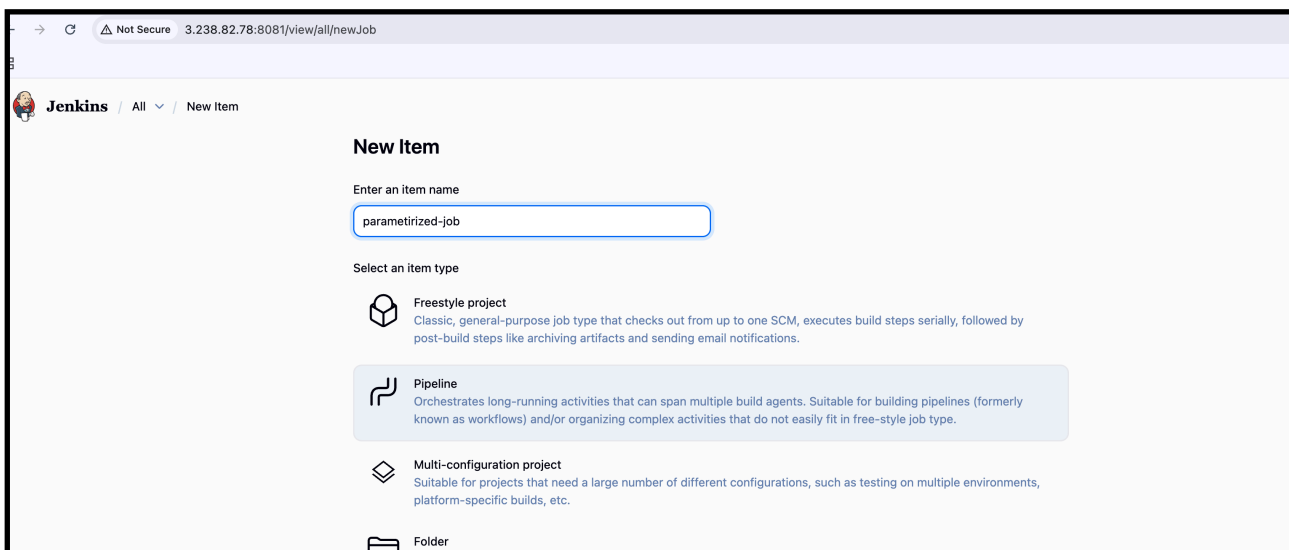
5) Create a parametrized job in jenkins.

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

Creating a Parameterized Jenkins Pipeline

Step 1. Start a New Pipeline Job

1. In Jenkins, click **New Item**.
2. Name your job and select **Pipeline**, then click **OK**.



Step 2: Add Parameters in Jenkins

1. Go to **Jenkins Dashboard** → **Your Job** → **Configure**.
2. Check **This project is parameterized**.
3. Add the following parameters:

The screenshot shows the Jenkins Configuration page for a job named 'parametrized-job'. The left sidebar contains a 'Configure' section with four tabs: 'General' (selected), 'Triggers', 'Pipeline', and 'Advanced'. The main area is titled 'General' and contains a 'Description' text area, a 'Plain text' link, and a list of checkboxes. The checkbox 'This project is parameterized' is checked. Below this, a 'String Parameter' section is visible, showing a 'Name' field and a 'Default Value' field.

Jenkins / parametrized-job / Configuration

Configure

- General
- Triggers
- Pipeline
- Advanced

General

Description

Plain text [Preview](#)

- ☐ Enable project-based security
- ☐ Discard old builds ?
- ☐ Do not allow concurrent builds
- ☐ Do not allow the pipeline to resume if the controller restarts
- ☐ GitHub project
- ☐ Pipeline speed/durability override ?
- ☐ Preserve stashes from completed builds ?
- ☒ This project is parameterized ?

String Parameter ?

Name ?

Default Value ?

1. String Parameter

- **Name:** GIT_BRANCH
- **Default Value:** main
- **Description:** Git branch to build from

☒ This project is parameterized ?

String Parameter ?

Name ?

GIT_BRANCH

Default Value ?

main

Description ?

Git branch to build from

Plain text

[Preview](#)

☐ Trim the string ?

2. String Parameter

- **Name:** GIT_URL
- **Default Value:** https://github.com/sunil-th/sunil-app.git
- **Description:** Git repository URL

String Parameter ?

Name ?

GIT_URL

Default Value ?

https://github.com/sunil-th/sunil-app.git

Description ?

Git repository URL

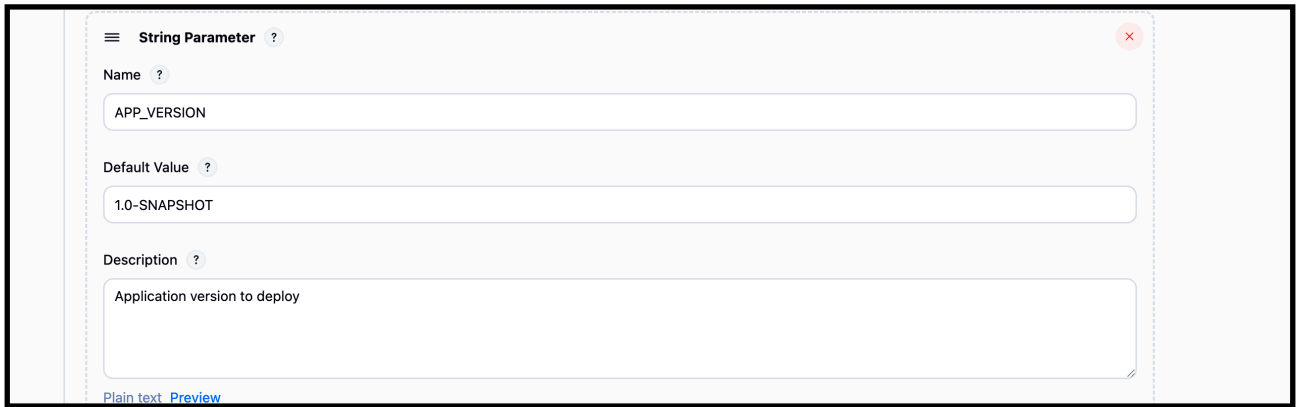
Plain text

[Preview](#)

☐ Trim the string ?

3. String Parameter

- **Name:** APP_VERSION
- **Default Value:** 1.0
- **Description:** Application version to deploy



String Parameter ?

Name ?

APP_VERSION

Default Value ?

1.0-SNAPSHOT

Description ?

Application version to deploy

Plain text Preview

4. String Parameter

- **Name:** NEXUS_REPO
- **Default Value:** hiring-app
- **Description:** Nexus repository name



String Parameter ?

Name ?

NEXUS_REPO

Default Value ?

waseem-app

Description ?

Nexus repository name

Plain text Preview

5. String Parameter

- **Name:** SOURCE_PATH
- **Default Value:** src/main/java
- **Description:** Path to source files (for SonarQube)

String Parameter ?

Name ?

SOURCE_PATH

Default Value ?

src/main/java

Description ?

Path to source files (for SonarQube)

Plain text [Preview](#)

☐ Trim the string ?

6. String Parameter

- **Name:** SLACK_MESSAGE
- **Default Value:** ☒ Build and deployment completed successfully
- **Description:** Slack notification message

String Parameter ?

Name ?

SLACK_MESSAGE

Default Value ?

☒ Build and deployment completed successfully

Description ?

Slack notification message

Plain text [Preview](#)

☐ Trim the string ?

7. Choice Parameter

- **Name:** MAVEN_GOAL
- **Choices:**
 - clean install
 - clean package
 - clean verify
- **Description:** Maven build goal

Choice Parameter ?

Name ?

MAVEN_GOAL

Choices ?

clean install
clean package
clean verify

Description ?

Maven build goal

Plain text

Preview

Add Parameter

8. String Parameter

- **Name:** TOMCAT_CONTEXT_PATH
- **Default Value:** /waseem-app
- **Description:** Tomcat app context path

String Parameter ?

Name ?

TOMCAT_CONTEXT_PATH

Default Value ?

/waseem-app

Description ?

Tomcat app context path

Step 2: Updated Scripted Pipeline Code



```
node {
```

```
def GIT_URL = 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
```

```
def GIT_BRANCH = 'feature1'
```

```
def WAR_FILE = ""
```

```
try {
```

```
stage('Git Clone') {
```

```
git branch: GIT_BRANCH, url: GIT_URL
```

```
}
```

```
stage('Maven Compilation') {
```

```
def mvnHome = tool 'MVN_HOME'
```

```
sh """
```

```
${mvnHome}/bin/mvn clean install
```

```
"""
```

```
}
```



```

stage('Detect WAR File') {
    WAR_FILE = sh(
        script: "ls target/*.war | head -n 1",
        returnStdout: true
    ).trim()

    echo "Detected WAR: ${WAR_FILE}"
}

```

```

stage('SonarQube Integration') {
    def scannerHome = tool 'sonar_scanner'

    withSonarQubeEnv('sonarqube-server1') {
        sh """
        ${scannerHome}/bin/sonar-scanner \
        -Dsonar.projectKey=Sabear \
        -Dsonar.projectName=Sabear \
        -Dsonar.projectVersion=1.0 \
        -Dsonar.sources=src \
        -Dsonar.java.binaries=target/**/WEB-INF/classes
        """
    }
}

```

```

stage('Nexus Artifactory') {
    nexusArtifactUploader(
        nexusVersion: 'nexus3',
        protocol: 'http',
        nexusUrl: '13.221.106.196:8081',
        repository: 'hiring-app',
        credentialsId: 'nexus-user',
        groupId: 'in.javahome',
        version: env.BUILD_NUMBER,
        artifacts: [[
            artifactId: 'SimpleCustomerApp',
            classifier: "",

```

```
        file: WAR_FILE,
        type: 'war'
    ]
)
}
```

```
stage('Deploy On Tomcat') {
    deploy adapters: [
        tomcat9(
            credentialsId: 'tomcat_credentials',
            url: 'http://44.204.98.178:8080'
        )
    ],
    contextPath: 'SimpleCustomerApp',
    war: WAR_FILE
}
```

```
stage('Slack Notification') {
    slackSend(
        channel: '#jenkins-integration',
        color: 'good',
        message: "✅ BUILD SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    )
}
```

```
} catch (err) {
```

```
    slackSend(
        channel: '#jenkins-integration',
        color: 'danger',
        message: "❌ BUILD FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    )
```

```
    throw err
```

```
}
```

```
}
```

Step 3: Save and Build with Parameter

The screenshot shows the Jenkins configuration page for a pipeline named 'parametrized-job'. The left sidebar contains navigation links: Status, Changes, Build with Parameters, Configure, Delete Pipeline, Stages, Rename, Pipeline Syntax, and Builds. The main area is titled 'Pipeline parametrized-job' and includes a note: 'This build requires parameters:'. Below this, several parameters are configured in text input fields: GIT_BRANCH (main), GIT_URL (https://github.com/sunil-th/sunil-app.git), APP_VERSION (1.0-SNAPSHOT), NEXUS_REPO (waseem-app), SOURCE_PATH (src/main/java), and SLACK_MESSAGE (Build and deployment completed successfully). A 'Builds' section on the left indicates 'No builds'.

Jenkins / parametrized-job

Pipeline parametrized-job

This build requires parameters:

GIT_BRANCH
Git branch to build from
main

GIT_URL
Git repository URL
https://github.com/sunil-th/sunil-app.git

APP_VERSION
Application version to deploy
1.0-SNAPSHOT

NEXUS_REPO
Nexus repository name
waseem-app

SOURCE_PATH
Path to source files (for SonarQube)
src/main/java

SLACK_MESSAGE
Slack notification message
☒ Build and deployment completed successfully

Builds
No builds

The screenshot shows the Jenkins job page for 'parametrized-job'. The left sidebar contains navigation links: Status, Changes, Build with Parameters, Configure, Delete Pipeline, Stages, Rename, Pipeline Syntax, and Builds. The main area is titled 'parametrized-job' and includes a 'Permalinks' section with a link: 'Last build (#1), 0.27 sec ago'. Below this, a 'Builds' section shows a list of builds. The first build is '#1' with a status of 'Success' and a time of '2:46 PM'.

Jenkins / parametrized-job

parametrized-job

Permalinks

- Last build (#1), 0.27 sec ago

Builds

Filter

Today

- #1 2:46 PM



Jenkins / parametrized-job



Status

</> Changes



Build with Parameters



Configure



Delete Pipeline



SonarQube



Stages



Rename



Pipeline Syntax



parametrized-job

SonarQube Quality Gate

Sabear **Passed**

server-side processing: **Success**

Permalinks

- [Last build \(#1\), 34 sec ago](#)
- [Last stable build \(#1\), 34 sec ago](#) ▾
- [Last successful build \(#1\), 34 sec ago](#)
- [Last completed build \(#1\), 34 sec ago](#)

Builds



Filter



Today



#1 2:46 PM



Jenkins / parametrized-job ▾ / Stages

Stages

January 3, 2026



#1

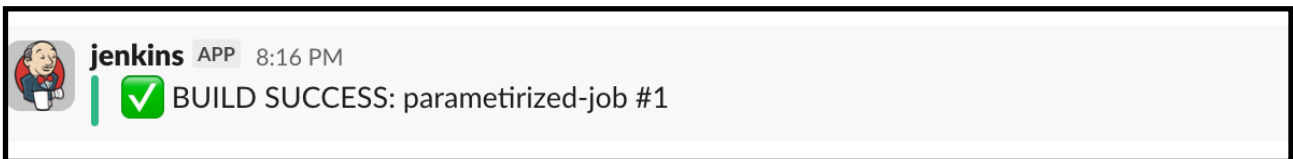
20:16 - 12s



How It Works

- When you click **Build with Parameters**, Jenkins will ask you to enter:
 - Branch, Repo URL, App Version, Nexus Repo name, Maven goal, Tomcat context, Slack details.
- Pipeline runs with those values.
- Example: you can build branch with feature-1.1 or master without touching the pipeline code.
- You can change Slack message on each run if needed.

Slack



6) Setup one slave machine for jenkins.

Jenkins Master–Slave Configuration

Steps to be done on slave

Step-1 : Create an ec2 –slave

Login to slave machine..

Install Java on slave

```
sudo dnf install -y java-17-amazon-corretto
```

```
#
#####
~\  #####
~~ \#####\
~~  \###|
~~   \#/
~~    v~'  ->
~~~~
~~ .-
~~ /_
~~ /m/ '  _/

Amazon Linux 2023

https://aws.amazon.com/linux/amazon-linux-2023

Last login: Sun Jan  4 08:48:19 2026 from 18.206.107.27
[ec2-user@ip-172-31-6-50 ~]$ sudo -i
[root@ip-172-31-6-50 ~]# sudo dnf install -y java-17-amazon-corretto
```

Check the java version

Java -version

Create a folder **jenkinsremote** folder

And give permissions to that folder

Chmod 777 jenkinsremote

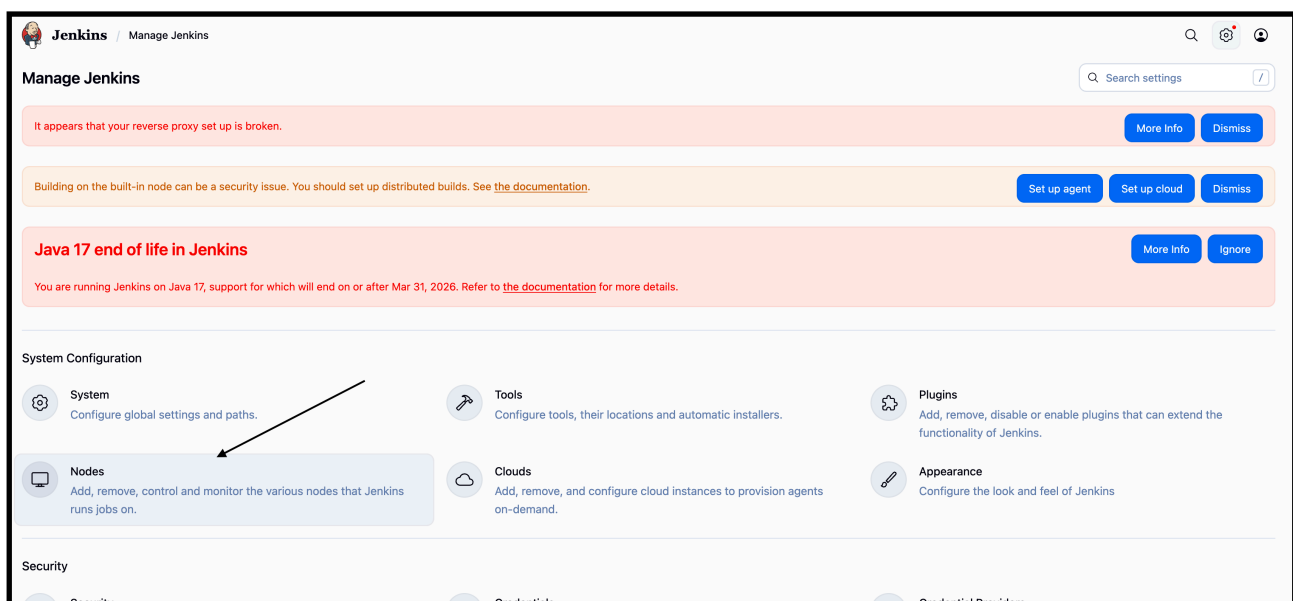
and Copy the pwd **/home/ec2-user/jenkinsremote**

```
[ec2-user@ip-172-31-6-50 ~]$ mkdir jenkinsremote
[ec2-user@ip-172-31-6-50 ~]$ ls
jenkinsremote
[ec2-user@ip-172-31-6-50 ~]$ chmod 777 jenkinsremote/
[ec2-user@ip-172-31-6-50 ~]$ ll
total 0
drwxrwxrwx. 2 ec2-user ec2-user 6 Jan  4 09:10 jenkinsremote
[ec2-user@ip-172-31-6-50 ~]$ cd jenkinsremote/
[ec2-user@ip-172-31-6-50 jenkinsremote]$ pwd
/home/ec2-user/jenkinsremote
[ec2-user@ip-172-31-6-50 jenkinsremote]$
```

i-0bf14b0eb013d170a (jenkins-slave)

Configure Node in Jenkins Master

1. Go to Jenkins Dashboard → Manage Jenkins → Nodes and Clouds → New Node
2. Enter node name (e.g., slave-1) → Select Permanent Agent → OK



← → ↻ Not Secure 44.210.70.29:8081/manage/computer/new

Jenkins / Manage Jenkins ▾ / Nodes ▾ / New node

New node

Node name

slave1

Type

☒ Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

3. Configure:

- **Remote root directory:** (/home/ec2-user/jenkinsremote)
- **Labels:** give a label (e.g., linux, build) to target jobs **(I have given java)**
- **Number of executors** → usually 1 (how many jobs can run in parallel on this agent).

Jenkins / Manage Jenkins ▾ / Nodes ▾

Name ?

slave1

Description ?

slave1

Plain text [Preview](#)

Number of executors ?

2

Remote root directory ?

/home/ec2-user/jenkinsremote

Labels ?

Add credentials (Jenkins user SSH key)

SSH Username with private key

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

ID ?

slave-1

Description ?

slave-1

Username

ec2-user

☐ Treat username as secret ?

Private Key

☒ Enter directly

Key

-----BEGIN RSA PRIVATE KEY-----
MIIeowIBAAKCAQEAmW/3d1drVMDkP+di2yFZ4uXCacGBkea2l+809g+4Qz7x1Kc
320sRVPUq4PrCbdIJTLvX5YzJRdBtK9SKDR9RYy+wfsW225gi3cobviXvW0zFQmZ
8Ha9MnxUFd76CUnyiscxhpfvFvtWU+CP2+lnZUnCCOpCLT7o+UaKqhD0mer/ts
xT1bDQ2tHufIGSybXLRzv758UHKZpfm39ryR7GUZIdPdowRH4e/dQ+svwU2PjEY9
qnchpSDC0/Yty3KfWHbkokeriLqc9L/4Rosn4+ZBrAm1sIBw1QcLwJ7Q00WBw1mU
U26zVegYE1HPg646lsIHL6A820o3/IsjPDYt3wIDAQABAoIBAQCCSrzGH3qpZiRa

Now go to node configuration and add **launch agent ssh**

Enter public /IP of ec2 slave

OR Launch agent via Java Web Start (JNLP) if SSH not possible

Usage → Choose Use this node as much as possible.

Use this node as much as possible

Launch method ?

Launch agents via SSH

Host ?

3.239.151.14

Credentials ?

ec2-user (slave-1)

+ Add

Host Key Verification Strategy ?

Non verifying Verification Strategy

Advanced

Availability ?

Keep this agent online as much as possible

Modify the disk space threshold

Test Connection

- Click **Save** → Jenkins will try to connect.
- Check node status in **Build Executor Status** (bottom left).
- If successful, node shows as **online**.

Nodes

+ New Node

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	R
	Built-In Node	Linux (amd64)	In sync	16.21 GiB	 0 B	3.80 GiB	R
	slave1	Linux (amd64)	In sync	17.98 GiB	 0 B	 458.26 MiB	R
Data obtained		1 min 19 sec	1 min 19 sec	1 min 19 sec	1 min 19 sec	1 min 19 sec	

Icon: S M L

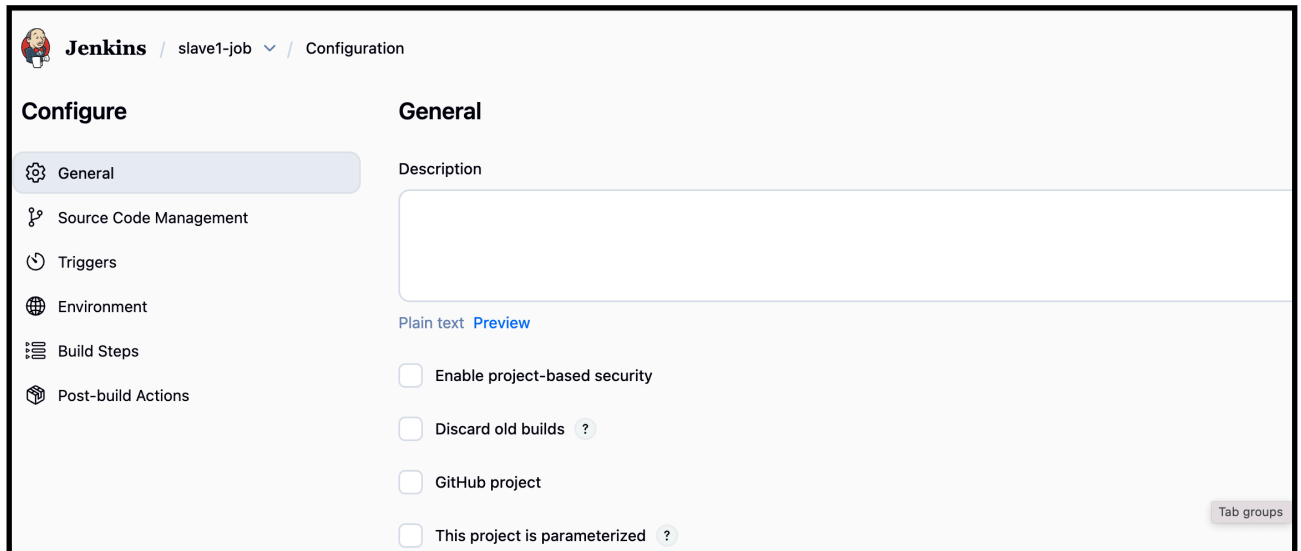
—> check log

Jenkins / Nodes / slave1 / Log

```
gawkpath_prepend ()
{
    unset AWKPATH;
    export AWKPATH=`gawk 'BEGIN {print ENVIRON["AWKPATH"]}'`
}
gawkpath_prepend ()
{
    [ -z "$AWKPATH" ] && AWKPATH=`gawk 'BEGIN {print ENVIRON["AWKPATH"]}'`;
    export AWKPATH="$*: $AWKPATH"
}
which ()
{
    ( alias;
    eval ${which_declare} ) | /usr/bin/which --tty-only --read-alias --read-functions --show-tilde --show-dot "$@"
}
[01/04/26 09:27:20] [SSH] Starting sftp client.
[01/04/26 09:27:20] [SSH] Copying latest remoting.jar...
[01/04/26 09:27:20] [SSH] Copied 1,398,060 bytes.
Expanded the channel window size to 4MB
[01/04/26 09:27:20] [SSH] Starting agent process: cd "/home/ec2-user/jenkinsremote" && java -jar remoting.jar -workDir
jar-cache /home/ec2-user/jenkinsremote/remoting/jarCache
Jan 04, 2026 9:27:20 AM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/ec2-user/jenkinsremote/remoting as a remoting work directory
Jan 04, 2026 9:27:20 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/ec2-user/jenkinsremote/remoting
<===[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3327.v868139a_d00e0
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

Run a Test Job

- Create a simple job and restrict it to the new agent by **label**.

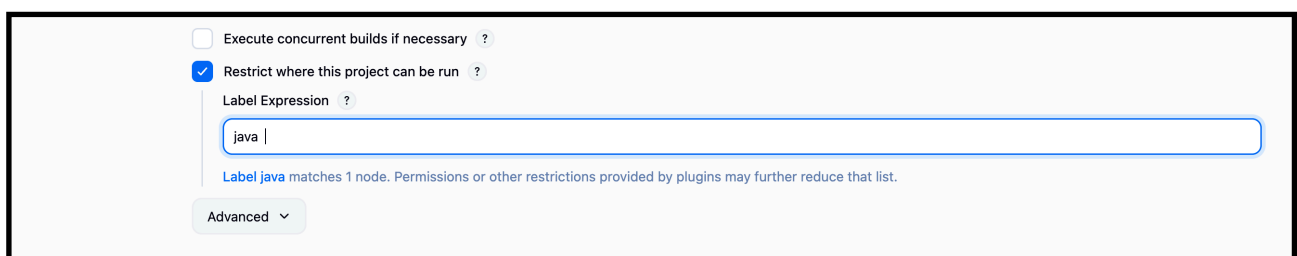


The image shows the Jenkins Configuration page for a job named 'slave1-job'. The breadcrumb navigation at the top reads 'Jenkins / slave1-job / Configuration'. On the left, under the 'Configure' heading, there is a sidebar with several tabs: 'General' (selected), 'Source Code Management', 'Triggers', 'Environment', 'Build Steps', and 'Post-build Actions'. The 'General' tab is active, showing a 'Description' text area and a 'Plain text Preview' link. Below these are four unchecked checkboxes: 'Enable project-based security', 'Discard old builds' (with a help icon), 'GitHub project', and 'This project is parameterized' (with a help icon). A 'Tab groups' button is located in the bottom right corner of the configuration area.

Give —> restrict where this project can be run

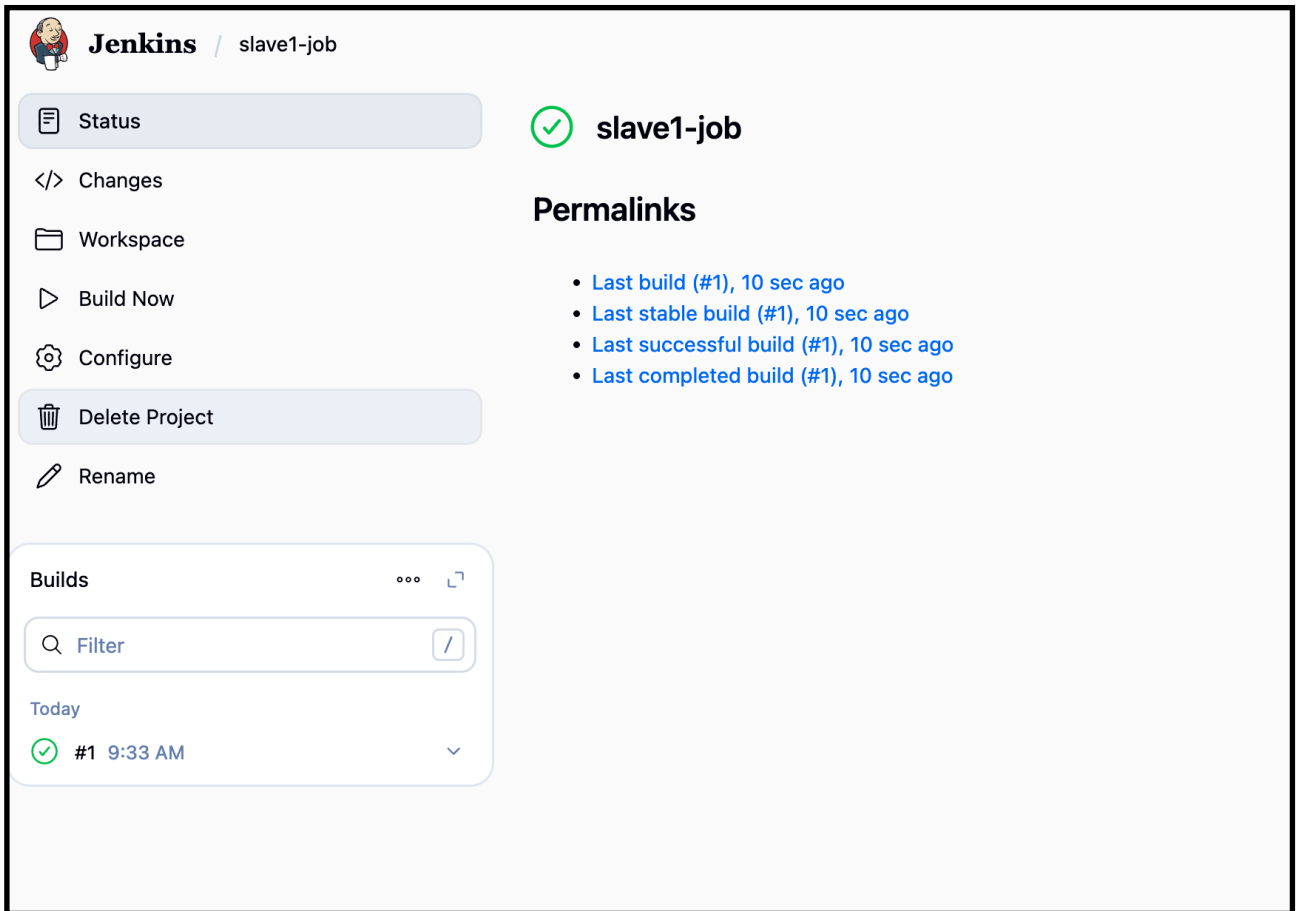
—> in label expression give **java**

—> and click save



The image shows a close-up of the 'Restrict where this project can be run' section in the Jenkins configuration. The checkbox is checked. Below it, the 'Label Expression' field contains the text 'java'. A blue link below the field states: 'Label java matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.' At the bottom of this section is an 'Advanced' button with a dropdown arrow.

—> now click build now and check the log status



The screenshot shows the Jenkins web interface for a project named 'slave1-job'. On the left is a sidebar with navigation links: Status (selected), Changes, Workspace, Build Now, Configure, Delete Project, and Rename. The main area displays a green checkmark icon and the project name 'slave1-job'. Below this is a 'Permalinks' section with four links: 'Last build (#1), 10 sec ago', 'Last stable build (#1), 10 sec ago', 'Last successful build (#1), 10 sec ago', and 'Last completed build (#1), 10 sec ago'. At the bottom, a 'Builds' section shows a single build, '#1', with a green checkmark icon and the time '9:33 AM'.



The screenshot shows the Jenkins web interface for the 'slave1-job' build #1, specifically the 'Console Output' view. The left sidebar includes links for Status, Changes, Console Output (selected), Edit Build Information, Delete build '#1', and Timings. The main area displays the console output with a green checkmark icon and the title 'Console Output'. At the top right of the console output area are buttons for 'Download', 'Copy', and 'View as plain text'. The console output text reads: 'Started by user waseem akram', 'Running as SYSTEM', 'Building remotely on slave1 (java) in workspace /home/ec2-user/jenkinsremote/workspace/slave1-job', and 'Finished: SUCCESS'.

—> by this Jenkins slave is connected

==> for conforming the connection go to your slave instance

```
[ec2-user@ip-172-31-6-50 jenkinsremote]$ pwd
/home/ec2-user/jenkinsremote
[ec2-user@ip-172-31-6-50 jenkinsremote]$ ls
remoting  remoting.jar  workspace
[ec2-user@ip-172-31-6-50 jenkinsremote]$ cd workspace/
[ec2-user@ip-172-31-6-50 workspace]$ ls
slave1-job
[ec2-user@ip-172-31-6-50 workspace]$ cd slave1-job/
```

—> and open the directory where we have given the path in node

/home/ec2-user/jenkinsremote

—> ls

—> cd workspace

—> ls

We are able to have the slave-job which we created by this slave node #slave1-job